

LeetCode Weekly Contest Prep — Core Content (C++)

This is actual teaching content, not a checklist. Read each section, understand the *why*, then go solve the practice problems listed.

0. Time Complexity — The One Thing That Decides Everything

LeetCode gives you constraints like $1 \leq n \leq 10^5$. This tells you what complexity is allowed, since judges give you $\sim 1-2$ seconds:

n (input size)	Allowed complexity	What that means
$n \leq 10$	$O(2^n)$, $O(n!)$	brute force / backtracking
$n \leq 20$	$O(2^n)$	bitmask DP
$n \leq 500$	$O(n^3)$	triple nested loops OK
$n \leq 5,000$	$O(n^2)$	double nested loops OK
$n \leq 10^6$	$O(n \log n)$	sorting, heaps, binary search
$n \leq 10^8$	$O(n)$	single pass
$n > 10^8$	$O(\log n)$ or $O(1)$	binary search, math formula

Read the constraints before you write a line of code. If $n \leq 10^5$ and your solution is $O(n^2)$, it will TLE even if the logic is correct.

C++ contest boilerplate — start every solution file with this so you're not fighting syntax mid-contest:

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    // your code
}
```

`#include <bits/stdc++.h>` pulls in essentially the whole standard library — vector, map, algorithm, etc. — so you never waste contest time hunting for the right header. `sync_with_stdio(false)` and `cin.tie(NULL)` speed up I/O, though on LeetCode itself you're writing a function, not `main()`, so this matters more for other judges.

1. Arrays & Vectors — The Baseline

Most Q1s are pure implementation: simulate what the problem describes, carefully, without a clever trick.

Key habits:

- Use `vector<int>` instead of raw arrays — dynamic sizing, `.size()`, range-based for loops.
- Use `for (auto& x : nums)` when you just need values; use index-based loops when you need positions.

- Watch off-by-one errors: `for (int i = 0; i < n; i++)` vs `for (int i = 0; i <= n; i++)`.
- Use `long long` instead of `int` whenever sums could exceed ~2 billion (a very common source of wrong answers in contests — “integer overflow”).

Example — Q1-style problem: > Given an array, return the count of elements strictly greater than the average.

```
int countGreaterThanAverage(vector<int>& nums) {
    double sum = 0;
    for (int x : nums) sum += x;
    double avg = sum / nums.size();

    int count = 0;
    for (int x : nums) {
        if (x > avg) count++;
    }
    return count;
}
```

No DSA trick here — just correct, careful implementation. A large fraction of contest points are lost to bugs in problems like this, not missing algorithms.

Practice: LeetCode Easy problems tagged “Array” and “String” — aim for 8-10 solved fluently (under 5 min each) before moving on.

2. Hash Map / Hash Set — $O(1)$ Lookup

The idea: if you find yourself writing a nested loop to check “does this value exist elsewhere?”, `unordered_map/unordered_set` turns $O(n^2)$ into $O(n)$.

Pattern — Two Sum:

```
vector<int> twoSum(vector<int>& nums, int target) {
    unordered_map<int, int> seen; // value -> index
    for (int i = 0; i < (int)nums.size(); i++) {
        int complement = target - nums[i];
        if (seen.count(complement)) {
            return {seen[complement], i};
        }
        seen[nums[i]] = i;
    }
    return {};
}
```

Why this works: instead of checking every pair ($O(n^2)$), you remember what you’ve seen and check membership in $O(1)$ average.

Pattern — Frequency counting:

```
unordered_map<int, int> freq;
for (int x : nums) freq[x]++;

// most frequent element
int best = nums[0];
for (auto& [val, count] : freq) {
    if (count > freq[best]) best = val;
}
```

Note on map vs unordered_map: `unordered_map` is $O(1)$ average but $O(n)$ worst case (rare, but LeetCode-adjacent judges have been known to have anti-hash-collision test cases); `map` is a sorted tree, $O(\log n)$ but

ordered. Default to `unordered_map/unordered_set` for speed unless you need sorted iteration.

When to reach for this: “have I seen this before”, counting occurrences, finding duplicates, grouping by a property (anagrams, equal sums, etc.)

Practice: Two Sum, Contains Duplicate, Valid Anagram, Group Anagrams, Top K Frequent Elements.

3. Two Pointers

The idea: when an array is sorted (or can be sorted), replace a nested loop with two pointers moving toward each other or together — $O(n)$ or $O(n \log n)$ instead of $O(n^2)$.

Pattern — Pair with target sum in sorted array:

```
vector<int> twoSumSorted(vector<int>& nums, int target) {
    int left = 0, right = nums.size() - 1;
    while (left < right) {
        int sum = nums[left] + nums[right];
        if (sum == target) {
            return {left, right};
        } else if (sum < target) {
            left++; // need a bigger sum
        } else {
            right--; // need a smaller sum
        }
    }
    return {};
}
```

Because the array is sorted, moving left right only increases the sum, and moving right left only decreases it — you never re-check a pair.

Pattern — Fast/slow pointers (remove duplicates in place):

```
int removeDuplicates(vector<int>& nums) {
    int slow = 0;
    for (int fast = 1; fast < (int)nums.size(); fast++) {
        if (nums[fast] != nums[slow]) {
            slow++;
            nums[slow] = nums[fast];
        }
    }
    return slow + 1;
}
```

When to reach for this: sorted array + pair/triplet search, in-place array modification, merging two sorted arrays, palindrome checks.

Practice: Two Sum II, 3Sum, Container With Most Water, Valid Palindrome, Merge Sorted Array.

4. Prefix Sums

The idea: if repeatedly asked “sum of elements from index i to j ”, precompute a running total once, then answer each query in $O(1)$.

```
vector<long long> buildPrefix(vector<int>& nums) {
    int n = nums.size();
    vector<long long> prefix(n + 1, 0);
```

```

    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + nums[i];
    }
    return prefix;
}

// sum of nums[i..j] inclusive:
long long rangeSum(vector<long long>& prefix, int i, int j) {
    return prefix[j + 1] - prefix[i];
}

```

Extension — Subarray Sum Equals K (prefix sum + hashmap combo):

```

int subarraySum(vector<int>& nums, int k) {
    int count = 0;
    long long prefixSum = 0;
    unordered_map<long long, int> seen; // prefix sum -> times seen
    seen[0] = 1;
    for (int x : nums) {
        prefixSum += x;
        if (seen.count(prefixSum - k)) {
            count += seen[prefixSum - k];
        }
        seen[prefixSum]++;
    }
    return count;
}

```

This combo (prefix sum + hashmap) is a very common Q2/Q3 trick — recognize it whenever the problem says “subarray with sum/property X”.

Practice: Range Sum Query, Subarray Sum Equals K, Product of Array Except Self.

5. Sliding Window

The idea: for problems about contiguous subarrays/substrings satisfying some condition, maintain a window [left, right] and slide it, adjusting as you go — O(n) instead of checking every subarray.

Fixed-size window (max sum of subarray of size k):

```

int maxSumSubarray(vector<int>& nums, int k) {
    int windowSum = 0;
    for (int i = 0; i < k; i++) windowSum += nums[i];

    int best = windowSum;
    for (int i = k; i < (int)nums.size(); i++) {
        windowSum += nums[i] - nums[i - k]; // add new, remove old
        best = max(best, windowSum);
    }
    return best;
}

```

Variable-size window (longest substring without repeating characters):

```

int lengthOfLongestSubstring(string s) {
    unordered_set<char> seen;
    int left = 0, best = 0;
    for (int right = 0; right < (int)s.size(); right++) {
        while (seen.count(s[right])) {
            seen.erase(s[left]);
        }
    }
}

```

```

        left++;
    }
    seen.insert(s[right]);
    best = max(best, right - left + 1);
}
return best;
}

```

How to recognize this pattern: the problem asks for something about a contiguous run — “longest”, “shortest”, “maximum”, “at most K distinct” — and the condition is monotonic (shrinking an invalid window can only help).

Practice: Longest Substring Without Repeating Characters, Minimum Size Subarray Sum, Longest Substring with At Most K Distinct Characters, Max Consecutive Ones III.

6. Binary Search (Beyond “Find X in Sorted Array”)

Standard binary search:

```

int binarySearch(vector<int>& nums, int target) {
    int lo = 0, hi = nums.size() - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // avoids overflow vs
(lo+hi)/2
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}

```

Note $lo + (hi - lo) / 2$ instead of $(lo + hi) / 2$ — the latter can overflow int when lo and hi are both large. Good habit to build now.

The contest-relevant extension: “binary search on the answer.”

Many Q3/Q4 problems ask you to *minimize the maximum* or *maximize the minimum* of something. Whenever you see that phrasing, and “if X works, anything looser than X also works” (monotonicity), binary search the answer space itself.

Pattern — Minimum capacity to ship packages within D days:

```

bool canShip(vector<int>& weights, int days, int capacity) {
    int dayCount = 1;
    long long current = 0;
    for (int w : weights) {
        if (current + w > capacity) {
            dayCount++;
            current = 0;
        }
        current += w;
    }
    return dayCount <= days;
}

int shipWithinDays(vector<int>& weights, int days) {
    int lo = *max_element(weights.begin(), weights.end());
    int hi = accumulate(weights.begin(), weights.end(), 0);
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (canShip(weights, days, mid)) {

```

```

        hi = mid;      // mid works, try smaller
    } else {
        lo = mid + 1;  // mid too small, need bigger
    }
}
return lo;
}

```

How to recognize this pattern: phrases like “minimum possible maximum”, “smallest value such that...”, “maximum such that at least/at most K”. Ask: “if I guess an answer, can I check feasibility in $O(n)$?” If yes → binary search on the answer.

Practice: Binary Search, Search in Rotated Sorted Array, Koko Eating Bananas, Capacity To Ship Packages Within D Days.

7. Greedy

The idea: at each step, make the locally best choice, trusting it leads to a globally optimal solution. Only works when the problem structure supports it (an exchange argument would justify it) — greedy is more about pattern recognition than a fixed template.

Pattern — Interval scheduling (max non-overlapping intervals):

```

int maxNonOverlapping(vector<vector<int>>& intervals) {
    sort(intervals.begin(), intervals.end(),
        [](vector<int>& a, vector<int>& b) { return a[1] < b[1];
}); // sort by END time

    int count = 0;
    long long lastEnd = LLONG_MIN;
    for (auto& interval : intervals) {
        if (interval[0] >= lastEnd) {
            count++;
            lastEnd = interval[1];
        }
    }
    return count;
}

```

Why sort by end time: picking the interval that finishes earliest always leaves the most room for future intervals — the classic greedy exchange argument.

Pattern — Assign cookies (two-pointer greedy on sorted arrays):

```

int findContentChildren(vector<int>& greed, vector<int>& cookies) {
    sort(greed.begin(), greed.end());
    sort(cookies.begin(), cookies.end());
    int i = 0, j = 0;
    while (i < (int)greed.size() && j < (int)cookies.size()) {
        if (cookies[j] >= greed[i]) i++;
        j++;
    }
    return i;
}

```

When to reach for this: “maximum number of non-overlapping...”, “minimum number to cover all...”, scheduling/interval problems, problems where sorting by some criterion makes the choice obvious.

Practice: Assign Cookies, Non-overlapping Intervals, Jump Game, Gas Station.

8. Graphs — BFS & DFS Basics

Representation (adjacency list):

```
vector<vector<int>> buildGraph(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph(n);
    for (auto& e : edges) {
        graph[e[0]].push_back(e[1]);
        graph[e[1]].push_back(e[0]); // omit this line if directed
    }
    return graph;
}
```

BFS — shortest path / level order (unweighted graphs):

```
vector<int> bfs(vector<vector<int>>& graph, int start) {
    int n = graph.size();
    vector<int> dist(n, -1);
    queue<int> q;
    dist[start] = 0;
    q.push(start);

    while (!q.empty()) {
        int node = q.front(); q.pop();
        for (int neighbor : graph[node]) {
            if (dist[neighbor] == -1) {
                dist[neighbor] = dist[node] + 1;
                q.push(neighbor);
            }
        }
    }
    return dist;
}
```

Use BFS when: you need shortest path in an unweighted graph, or “minimum number of steps/moves” — often graphs in disguise (grid problems, word ladder).

DFS — connectivity / counting components:

```
void dfs(int node, vector<vector<int>>& graph, vector<bool>& visited) {
    visited[node] = true;
    for (int neighbor : graph[node]) {
        if (!visited[neighbor]) {
            dfs(neighbor, graph, visited);
        }
    }
}

int countComponents(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph = buildGraph(n, edges);
    vector<bool> visited(n, false);
    int components = 0;
    for (int node = 0; node < n; node++) {
        if (!visited[node]) {
            dfs(node, graph, visited);
            components++;
        }
    }
    return components;
}
```

Use DFS when: exploring all paths, detecting cycles, counting connected components, or the problem is naturally recursive (e.g., “number of islands” on a grid).

Grids are graphs: a 2D grid problem (“number of islands”, “max area of island”) is just a graph where each cell connects to its 4 (or 8) neighbors. Recognizing this turns a confusing grid problem into a familiar BFS/DFS template.

Practice: Number of Islands, Flood Fill, Clone Graph, Course Schedule (cycle detection), Rotting Oranges (BFS multi-source).

9. Dynamic Programming — Just Enough to Start

DP is deep, but contest Q3/early-Q4 DP is usually **1D DP**: define what $dp[i]$ means, find the recurrence, handle base cases.

Pattern — Climbing Stairs:

```
int climbStairs(int n) {
    if (n <= 2) return n;
    vector<int> dp(n + 1);
    dp[1] = 1; dp[2] = 2;
    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

$dp[i]$ = number of ways to reach step i . You can only arrive from step $i-1$ or $i-2$, so $dp[i] = dp[i-1] + dp[i-2]$.

Pattern — House Robber (DP with a choice at each step, $O(1)$ space):

```
int rob(vector<int>& nums) {
    int prev = 0, curr = 0;
    for (int x : nums) {
        int temp = max(curr, prev + x);
        prev = curr;
        curr = temp;
    }
    return curr;
}
```

$dp[i]$ = max money robbable using houses $0..i$. At each house: skip it ($dp[i-1]$) or rob it ($dp[i-2] + \text{nums}[i]$). Take the max. Rolling two variables instead of a full array saves memory — a nice habit for contests.

How to recognize 1D DP: the problem asks for a count/min/max over a sequence, and the answer for position i can be built from answers at smaller positions. Ask: “if I knew the answer for everything before position i , could I compute position i ?”

Practice: Climbing Stairs, House Robber, Maximum Subarray (Kadane’s), Coin Change.

10. Pattern Recognition Cheat Sheet

Signal in the problem	Likely pattern
“have you seen this before”, counting, grouping	Hash Map/Set
Sorted array, pair/triplet with	Two Pointers

target	
Range sum queries, subarray sum	Prefix Sum
"longest/shortest contiguous subarray/substring satisfying..."	Sliding Window
"minimum possible maximum" / "maximum possible minimum"	Binary Search on Answer
"maximum non-overlapping", scheduling, sort-then-decide	Greedy
Grid with connectivity, "number of islands/regions"	BFS/DFS on grid
"minimum steps/moves" in unweighted graph	BFS
Answer at position i depends on answer at i-1, i-2...	1D DP

C++-Specific Contest Pitfalls

These cost more contest points than missing algorithms:

- **Integer overflow:** int maxes out around 2.1 billion. Sums of arrays with n up to 10^5 and values up to 10^9 can overflow int — use long long. When in doubt, use long long.
 - **(lo + hi) / 2 overflow in binary search:** use $lo + (hi - lo) / 2$ instead.
 - **Off-by-one in loop bounds:** double check $<$ vs $\leq$, and $size() - 1$ vs $size()$.
 - **Uninitialized variables:** C++ does not zero-initialize local `int/vector<int>(n)` elements the way some languages do by default for global scope, but local `vector<int> v(n)` *does* zero-init — know which is which. Raw arrays (`int arr[n]`) do **not** auto-initialize.
 - **Passing large containers by value:** always pass `vector<int>&` (reference) not `vector<int>` (copy) to avoid silent $O(n)$ copies that add up.
 - **size() returns size_t (unsigned):** comparing `int i` to `nums.size()` in a loop can behave oddly if `i` goes negative. Cast with `(int)nums.size()` or declare loop variables as `size_t/int` consistently.
-

What to Do With This

1. Read one section fully, including the code — trace through it by hand on paper with a small example.
2. Solve 4-6 problems from that section's practice list on LeetCode (set your language to C++), without looking at the solution first (15-20 min each).
3. If stuck after 15-20 min, read the editorial, understand the gap, then move on.
4. On your last day before the contest, do a full **Virtual Contest** on an old Weekly Contest under real time pressure — this matters more than anything else on this list.

You don't need all 9 sections mastered to start. Sections 1-5 (Arrays, Hash Map, Two Pointers, Prefix Sum, Sliding Window) will let you solve Q1 and Q2 in most contests, and often Q3. Start there.